

BeautyBook

Códigos PlantUML — Todos os Diagramas

Projeto de Software · PUC Minas · 2026

1. DIAGRAMA DE CASO DE USO

```
@startuml
left to right direction
skinparam actorStyle awesome
```

```
actor Cliente
actor Funcionário
actor Administrador
```

```
rectangle BeautyBook {
    usecase "UC-01: Cadastrar cliente" as UC01
    usecase "UC-02: Realizar agendamento" as UC02
    usecase "UC-03: Cancelar agendamento" as UC03
    usecase "UC-04: Visualizar agenda" as UC04
    usecase "UC-05: Gerenciar serviços" as UC05
    usecase "UC-06: Visualizar relatórios" as UC06
    usecase "UC-07: Notificar cliente por e-mail" as UC07
    usecase "UC-08: Aplicar cupom de desconto" as UC08
    usecase "UC-09: Exportar relatório em PDF" as UC09
}
```

```
usecase "UC-10: Avaliar atendimento" as UC10
usecase "UC-11: Reagendar horário" as UC11
usecase "UC-12: Bloquear horário na agenda" as UC12
}
```

```
Cliente --> UC01
Cliente --> UC02
Cliente --> UC03
Cliente --> UC10
Cliente --> UC11
```

```
Funcionário --> UC04
Funcionário --> UC12
```

```
Administrador --> UC05
Administrador --> UC06
```

```
UC02 .> UC08 : <<extend>>
UC06 .> UC09 : <<extend>>
UC11 .> UC07 : <<include>>
UC03 .> UC07 : <<include>>
@enduml
```

2. DIAGRAMA DE CONTEXTO (C4)

```
@startuml
!include
https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4\_Context.puml
```

```
LAYOUT_WITH_LEGEND()
```

```
title Diagrama de Contexto - BeautyBook
```

```
Person(cliente, "Cliente", "Realiza agendamentos\nonline pelo sistema")
Person(administrador, "Administrador", "Gerencia serviços, funcionários\ne relatórios")
```

```
Person(funcionario, "Funcionário", "Visualiza e confirma agenda\nde atendimentos")
```

```
System(beautybook, "BeautyBook", "Frontend Web (Interface do\ncliente e painel administrativo)\nReact/TypeScript")
```

```
System_Ext(googleOAuth2, "Google OAuth2", "Autenticação social do cliente")
```

```
System_Ext(sendgrid, "SendGrid", "Envio de e-mails de confirmação e cancelamento")
```

```
System_Ext(twilio, "Twilio", "Envio de SMS de lembrete de agendamento")
```

```
System_Ext(stripe, "Stripe", "Processamento de pagamentos online")
```

```
SystemDb_Ext(db, "Banco de Dados", "Armazena clientes, agendamentos\ne serviços - PostgreSQL 15")
```

```
Rel(cliente, beautybook, "Autentica via", "OAuth2/OpenID")
```

```
Rel(administrador, beautybook, "Administra via", "HTTPS/Browser")
```

```
Rel(funcionario, beautybook, "Consulta via", "HTTPS/Browser")
```

```
Rel(beautybook, googleOAuth2, "Valida token para", "JWT/HTTPS")
```

```
Rel(beautybook, db, "Lê e escreve", "JDBC/SQL")
```

```
Rel(beautybook, sendgrid, "Envia notificações via", "SMTP/API")
```

```
Rel(beautybook, twilio, "Envia lembretes via", "API REST")
```

```
Rel(beautybook, stripe, "Processa pagamentos via", "API REST/TLS")
```

```
@enduml
```

3. DIAGRAMA DE COMPONENTES

```
@startuml
```

```
skinparam componentStyle rectangle
```

```
actor Cliente
```

```

actor Funcionário
actor Administrador

package "Frontend" {
  [UI Web]
}

package "Backend" {
  [API REST]
  interface "IAuth" as IAuth
  interface "IAgendamento" as IAgendamento
  interface "ICliente" as ICliente
  interface "IRelatorio" as IRelatorio
  interface "IEmail" as IEmail

  [AuthService]
  [AgendamentoService]
  [ClienteService]
  [RelatorioService]
  [EmailService]

  [API REST] -- IAuth
  [API REST] -- IAgendamento
  [API REST] -- ICliente
  [API REST] -- IRelatorio
  IAuth --> [AuthService]
  IAgendamento --> [AgendamentoService]
  ICliente --> [ClienteService]
  IRelatorio --> [RelatorioService]
  [EmailService] -- IEmail
  [AgendamentoService] ..> IEmail
}

package "Infraestrutura" {
  [MailSender]
  [JWTProvider]
  [PostgreSQL]
  [PDFGenerator]
}

Cliente --> [UI Web]
Funcionário --> [UI Web]

```

Administrador --> [UI Web]

[UI Web] --> [API REST] : HTTP/REST

[AuthService] --> [JWTProvider] : JWT

[EmailService] --> [MailSender] : TCP/IP

[AgendamentoService] --> [PostgreSQL] : TCP/IP

[ClienteService] --> [PostgreSQL] : TCP/IP

[RelatorioService] --> [PostgreSQL] : TCP/IP

[RelatorioService] --> [PDFGenerator] : IPDF

@enduml

4. DIAGRAMA DE IMPLANTAÇÃO

@startuml

```
node "CDN (AWS CloudFront)" {  
    artifact "Assets Estáticos"  
}
```

```
node "Cliente" {  
    component "React App (SPA)"  
}
```

```
node "Load Balancer (AWS ALB)" {  
    component "Application Load Balancer"  
}
```

```
node "Servidor de Aplicação (AWS EC2)" {  
    component "Spring Boot API - Java 17"  
    component "Spring Security (JWT)"  
}
```

```
node "Banco de Dados (AWS RDS)" {  
    database "PostgreSQL 15 (Primary)"  
    database "PostgreSQL 15 (Read Replica)"  
    "PostgreSQL 15 (Primary)" --> "PostgreSQL 15 (Read  
Replica)" : replicação  
}
```

```

node "Cache (AWS ElastiCache)" {
    component "Redis"
}

node "Serviços Externos" {
    component "SendGrid (E-mail)"
    component "Stripe (Pagamentos)"
    component "Google OAuth2"
}

"CDN (AWS CloudFront)" --> "Cliente" : serve assets
"Cliente" --> "Load Balancer (AWS ALB)" : HTTPS / REST
"Load Balancer (AWS ALB)" --> "Servidor de Aplicação (AWS EC2)" : HTTP interno
"Spring Boot API - Java 17" --> "Spring Security (JWT)" : valida requisição
"Spring Boot API - Java 17" --> "PostgreSQL 15 (Primary)" : JDBC escrita
"PostgreSQL 15 (Primary)" --> "Spring Boot API - Java 17" : JDBC leitura
"Spring Boot API - Java 17" --> "Redis" : cache
"Spring Boot API - Java 17" --> "SendGrid (E-mail)" : API REST
"Spring Boot API - Java 17" --> "Stripe (Pagamentos)" : API REST
"Spring Boot API - Java 17" --> "Google OAuth2" : OAuth2
@enduml

```

5. DIAGRAMAS DE SEQUÊNCIA

UC-01: Cadastrar Cliente

```

@startuml
title UC-01: Cadastrar Cliente

actor Cliente
participant ClienteController
participant ClienteRepository
participant "Sistema de Email"

```

```

== 1. Envio de Dados ==
Cliente -> ClienteController : 1.1 POST /clientes
(ClienteRequestDTO)
ClienteController -> ClienteController : 1.2
validarDados(dto)
ClienteController -> ClienteRepository : 1.3
existsByEmail(dto.email)
ClienteRepository --> ClienteController : 1.4 boolean

alt já cadastrado
  ClienteController --> Cliente : 1.5 "Email já cadastrado."
else disponível
  ClienteController -> ClienteRepository : 1.6 save(cliente)
  ClienteRepository --> ClienteController : 1.7 clienteSalvo

== 2. Confirmação ==
  ClienteController -> "Sistema de Email" : 2.1
enviarBoasVindas(cliente.email)
  ClienteController --> Cliente : 2.2 [e-mail] Boas-vindas
enviado
  ClienteController --> Cliente : 2.3 201 Created
(ClienteResponseDTO)
end
@enduml

```

UC-02: Realizar Agendamento

```

@startuml
title UC-02: Realizar Agendamento

actor Cliente
participant AgendamentoController
participant AgendamentoRepository
participant "Sistema de Email"

== 1. Seleção de Serviço e Horário ==
Cliente -> AgendamentoController : 1.1
selecionarServico(servicoId)
AgendamentoController --> Cliente : 1.2
listaFuncionariosDisponiveis

```

```

Cliente -> AgendamentoController : 1.3
selecionarHorario(funcionarioId, dataHora)
AgendamentoController -> AgendamentoRepository : 1.4
verificarDisponibilidade(funcionarioId, dataHora)
AgendamentoRepository --> AgendamentoController : 1.5
statusDisponibilidade

== 2. Confirmação do Agendamento ==
alt disponível
  Cliente -> AgendamentoController : 2.1
  realizarAgendamento(clienteId, servicoId, funcionarioId,
dataHora)
  AgendamentoController -> AgendamentoRepository : 2.2
  save(agendamento)
  AgendamentoRepository --> AgendamentoController : 2.3
  agendamentoSalvo
  AgendamentoController -> "Sistema de Email" : 2.4
  enviarConfirmacao(cliente, detalhesReserva)
  AgendamentoController --> Cliente : 2.5 [e-mail]
  Confirmação de agendamento enviada
  AgendamentoController --> Cliente : 2.6 "Agendamento
confirmado com sucesso."
else indisponível
  AgendamentoController --> Cliente : 2.7 "Horário já
reservado. Escolha outro horário."
end
@enduml

```

UC-03: Cancelar Agendamento

```

@startuml
title UC-03: Cancelar Agendamento

actor Cliente
participant AgendamentoController
participant AgendamentoRepository
participant "Sistema de Email"

== 1. Solicitação de Cancelamento ==

```



```

Cliente -> AgendamentoController : 1.1
cancelarAgendamento(agendamentoId)
AgendamentoController -> AgendamentoRepository : 1.2
findByIdAndClienteId(agendamentoId, clienteId)
AgendamentoRepository --> AgendamentoController : 1.3
agendamento
AgendamentoController -> AgendamentoController : 1.4
verificarAntecedenciaMinima(agendamento.dataHora)

== 2. Processamento ==
alt >= 2h
    AgendamentoController -> AgendamentoRepository : 2.1
    cancelarReserva(agendamentoId)
    AgendamentoRepository --> AgendamentoController : 2.2
    horarioLiberado
    AgendamentoController -> "Sistema de Email" : 2.3
    enviarCancelamento(cliente, detalhesCancelamento)
    AgendamentoController --> Cliente : 2.4 [e-mail/SMS]
    Cancelamento confirmado
    AgendamentoController --> Cliente : 2.5 "Reserva cancelada.
    Horário liberado."
else < 2h
    AgendamentoController --> Cliente : 2.6 "Cancelamento fora
    do prazo permitido (< 2h)."
end
@enduml

```

UC-04: Visualizar Agenda

```

@startuml
title UC-04: Visualizar Agenda

actor Funcionário
participant AgendaController
participant AgendaService
participant AgendamentoRepository

Funcionário -> AgendaController : GET /agenda?data={data}
AgendaController -> AgendaService :
buscarAgendaDia(funcionarioId, data)

```

```

AgendaService -> AgendamentoRepository :
findByFuncionarioIdAndData(funcionarioId, data)
AgendamentoRepository --> AgendaService : listaAgendamentos
AgendaService --> AgendaController :
List<AgendamentoResponseDTO>
AgendaController --> Funcionário : 200 OK
@enduml

```

UC-05: Gerenciar Serviços

```

@startuml
title UC-05: Gerenciar Serviços

actor Administrador
participant ServicoController
participant ServicoRepository

== 1. Cadastro de Serviço ==
Administrador -> ServicoController : 1.1 POST /servicos
(ServicoRequestDTO)
ServicoController -> ServicoController : 1.2
validarCampos(dto)

alt inválidos
    ServicoController --> Administrador : 1.3 400 Bad Request
else válidos
    ServicoController -> ServicoRepository : 1.4 save(servico)
    ServicoRepository --> ServicoController : 1.5 servicoSalvo

== 2. Confirmação ==
    ServicoController --> Administrador : 2.1 201 Created
    (ServicoResponseDTO)
end
@enduml

```

UC-06: Visualizar Relatórios

```

@startuml

```

title UC-06: Visualizar Relatórios

actor Administrador

participant RelatorioController

participant AgendamentoRepository

== 1. Geração do Relatório ==

Administrador -> RelatorioController : 1.1 GET

/relatorios?periodo={periodo}&tipo={tipo}

RelatorioController -> RelatorioController : 1.2

verificarAutenticacaoAdmin()

RelatorioController -> AgendamentoRepository : 1.3

findByPeriodo(dataInicio, dataFim)

AgendamentoRepository --> RelatorioController : 1.4

listaAgendamentos

== 2. Consolidação ==

RelatorioController -> RelatorioController : 2.1

consolidarMetricas(listaAgendamentos)

RelatorioController --> Administrador : 2.2 200 OK

(RelatorioResponseDTO)

@enduml

UC-07: Notificar Cliente por E-mail

@startuml

title UC-07: Notificar Cliente por E-mail

participant ServicoOrigem

participant EmailService

participant JavaMailSender

actor Destinatário

note over ServicoOrigem : Disparado por: UC-02, UC-03, UC-11

== 1. Montagem da Mensagem ==

ServicoOrigem -> EmailService : 1.1

enviarNotificacao(destinatario, assunto, corpo)

EmailService -> EmailService : 1.2

montarMensagem(destinatario, assunto, corpo)

```

== 2. Envio ==
opt não enviada
  EmailService -> JavaMailSender : 2.1 send(mimeMessage)
  JavaMailSender --> EmailService : 2.2 confirmacaoEntrega
  EmailService --> ServicoOrigem : 2.3 emailEnviadoComSucesso
  JavaMailSender -> Destinatário : 2.4 [e-mail] notificação
recebida
end
@enduml

```

UC-08: Aplicar Cupom de Desconto

```

@startuml
title UC-08: Aplicar Cupom de Desconto

actor Cliente
participant AgendamentoController
participant CupomService
participant CupomRepository

note over AgendamentoController : Chamado durante UC-02

== 1. Validação do Cupom ==
Cliente -> AgendamentoController : 1.1 POST /agendamentos
(dto + codigoCupom)
AgendamentoController -> CupomService : 1.2
aplicarCupom(codigoCupom, valorTotal)
CupomService -> CupomRepository : 1.3
findByCodigo(codigoCupom)
CupomRepository --> CupomService : 1.4 cupom

== 2. Aplicação do Desconto ==
alt inválido ou expirado
  CupomService --> AgendamentoController : 2.1 "Cupom
inválido ou expirado."
  AgendamentoController --> Cliente : 2.2 422 Unprocessable
Entity
else válido

```

```

    CupomService -> CupomService : 2.3 calcularDesconto(cupom,
valorTotal)
    CupomService -> CupomRepository : 2.4
marcarComoUtilizado(cupom.id)
    CupomRepository --> CupomService : 2.5 void
    CupomService --> AgendamentoController : 2.6
valorComDesconto
    AgendamentoController --> Cliente : 2.7 201 Created
(desconto aplicado)
end
@enduml

```

UC-09: Exportar Relatório em PDF

```

@startuml
title UC-09: Exportar Relatório em PDF

actor Administrador
participant RelatorioController
participant AgendamentoRepository
participant PDFGenerator

note over RelatorioController : Extensão de UC-06

== 1. Busca de Dados ==
Administrador -> RelatorioController : 1.1 GET
/relatorios/export?periodo={periodo}
RelatorioController -> AgendamentoRepository : 1.2
findByPeriodo(dataInicio, dataFim)
AgendamentoRepository --> RelatorioController : 1.3
listaAgendamentos
RelatorioController -> RelatorioController : 1.4
consolidarMetricas(listaAgendamentos)

== 2. Geração do PDF ==
RelatorioController -> PDFGenerator : 2.1
gerarPDF(relatorioDTO)
PDFGenerator --> RelatorioController : 2.2 arquivoPDF
RelatorioController --> Administrador : 2.3 200 OK
(application/pdf)

```

@enduml

UC-10: Avaliar Atendimento

@startuml

title UC-10: Avaliar Atendimento

actor Cliente

participant AvaliacaoController

participant AgendamentoRepository

participant AvaliacaoRepository

== 1. Validação do Atendimento ==

Cliente -> AvaliacaoController : 1.1 POST /avaliacoes
(AvaliacaoRequestDTO)

AvaliacaoController -> AgendamentoRepository : 1.2
findById(agendamentoId)

AgendamentoRepository --> AvaliacaoController : 1.3
agendamento

AvaliacaoController -> AvaliacaoController : 1.4
verificarStatusConcluido(agendamento)

== 2. Registro da Avaliação ==

alt não concluído

AvaliacaoController --> Cliente : 2.1 "Atendimento ainda
não concluído."

else concluído

AvaliacaoController -> AvaliacaoRepository : 2.2
save(avaliacao)

AvaliacaoRepository --> AvaliacaoController : 2.3
avaliacaoSalva

AvaliacaoController --> Cliente : 2.4 201 Created
(AvaliacaoResponseDTO)

end

@enduml

UC-11: Reagendar Horário

```

@startuml
title UC-11: Reagendar Horário

actor Cliente
participant AgendamentoController
participant AgendamentoRepository
participant "Sistema de Email"

== 1. Verificação de Disponibilidade ==
Cliente -> AgendamentoController : 1.1 PATCH
/agendamentos/{id}/reagendar (novaDataHora)
AgendamentoController -> AgendamentoRepository : 1.2
findByIdAndClienteId(agendamentoId, clienteId)
AgendamentoRepository --> AgendamentoController : 1.3
agendamento
AgendamentoController -> AgendamentoRepository : 1.4
verificarDisponibilidade(funcionarioId, novaDataHora)
AgendamentoRepository --> AgendamentoController : 1.5
statusDisponibilidade

== 2. Atualização ==
alt horário indisponível
    AgendamentoController --> Cliente : 2.1 "Horário
    indisponível. Escolha outro horário."
else horário disponível
    AgendamentoController -> AgendamentoRepository : 2.2
    updateDataHora(agendamentoId, novaDataHora)
    AgendamentoRepository --> AgendamentoController : 2.3
    agendamentoAtualizado
    AgendamentoController -> "Sistema de Email" : 2.4
    enviarConfirmacaoReagendamento(cliente.email, agendamento)
    AgendamentoController --> Cliente : 2.5 [e-mail]
    Confirmação do novo horário enviada
    AgendamentoController --> Cliente : 2.6 200 OK
    (AgendamentoResponseDTO)
end
@enduml

```

UC-12: Bloquear Horário na Agenda

```

@startuml
title UC-12: Bloquear Horário na Agenda

actor Funcionário
participant AgendaController
participant AgendamentoRepository
participant BloqueioRepository

== 1. Verificação de Conflitos ==
Funcionário -> AgendaController : 1.1 POST /agenda/bloqueios
(BloqueioRequestDTO)
AgendaController -> AgendamentoRepository : 1.2
verificarConflito(funcionarioId, dataInicio, dataFim)
AgendamentoRepository --> AgendaController : 1.3
statusConflito

== 2. Registro do Bloqueio ==
alt agendamento no período
  AgendaController --> Funcionário : 2.1 "Conflito
  encontrado. Há agendamento no período."
else livre
  AgendaController -> BloqueioRepository : 2.2 save(bloqueio)
  BloqueioRepository --> AgendaController : 2.3 bloqueioSalvo
  AgendaController --> Funcionário : 2.4 201 Created
  (BloqueioResponseDTO)
end
@enduml

```

6. DIAGRAMA GERAL DE SEQUÊNCIA

```

@startuml
title Diagrama de Sequência Geral - BeautyBook

actor Cliente
actor Funcionário
actor Administrador
participant "Sistema BeautyBook"
database "Banco de Dados"
participant Pagamento

```


participant Email

== Cliente ==

```
Cliente -> "Sistema BeautyBook" : login()
"Sistema BeautyBook" -> "Banco de Dados" : validarUsuario()
"Banco de Dados" --> "Sistema BeautyBook" : usuarioValido
```

```
Cliente -> "Sistema BeautyBook" : consultarHorarios()
"Sistema BeautyBook" -> "Banco de Dados" : buscarHorarios()
"Banco de Dados" --> "Sistema BeautyBook" :
horariosDisponiveis
```

```
Cliente -> "Sistema BeautyBook" : agendarServico()
"Sistema BeautyBook" -> "Banco de Dados" :
salvarAgendamento()
"Banco de Dados" --> "Sistema BeautyBook" :
agendamentoCriado
"Sistema BeautyBook" -> Pagamento : processarPagamento()
Pagamento --> "Sistema BeautyBook" : pagamentoAprovado
"Sistema BeautyBook" -> Email : enviarConfirmacao()
Email --> "Sistema BeautyBook" : emailEnviado
"Sistema BeautyBook" --> Cliente : agendamentoConfirmado()
```

```
Cliente -> "Sistema BeautyBook" : cancelarAgendamento()
"Sistema BeautyBook" -> "Banco de Dados" : atualizarStatus()
"Banco de Dados" --> "Sistema BeautyBook" :
cancelamentoRealizado
```

== Funcionário ==

```
Funcionário -> "Sistema BeautyBook" : visualizarAgenda()
"Sistema BeautyBook" -> "Banco de Dados" : consultarAgenda()
"Banco de Dados" --> "Sistema BeautyBook" : agendaDoDia
```

```
Funcionário -> "Sistema BeautyBook" : atualizarStatus()
"Sistema BeautyBook" -> "Banco de Dados" : salvarStatus()
"Banco de Dados" --> "Sistema BeautyBook" : statusAtualizado
```

== Administrador ==

```
Administrador -> "Sistema BeautyBook" : gerenciarServicos()
"Sistema BeautyBook" -> "Banco de Dados" : salvarServico()
"Banco de Dados" --> "Sistema BeautyBook" : servicoSalvo
```

```
Administrador -> "Sistema BeautyBook" : gerarRelatorio()
"Sistema BeautyBook" -> "Banco de Dados" : consultarDados()
"Banco de Dados" --> "Sistema BeautyBook" : dadosRelatorio
"Sistema BeautyBook" --> Administrador : relatorioGerado()
@enduml
```

7. DIAGRAMA DE ESTADOS

```
@startuml
title Diagrama de Transição de Estados - BeautyBook

[*] --> ClienteNaoCadastrado : início do sistema

state "ClienteNaoCadastrado\nautenticado = false" as
ClienteNaoCadastrado
state "ClienteCadastrado\ncontaAtiva = true" as
ClienteCadastrado
state "ClienteAutenticado\ntokenJWT = ativo" as
ClienteAutenticado
state "AgendamentoPendente\nstatus = pendente" as
AgendamentoPendente
state "AgendamentoConfirmado\nstatus = confirmado" as
AgendamentoConfirmado
state "AgendamentoCancelado\nstatus = cancelado" as
AgendamentoCancelado
state "AgendamentoConcluido\nstatus = concluido" as
AgendamentoConcluido
state "NotificacaoPendente\ntentativas = 0" as
NotificacaoPendente
state "NotificacaoEnviada\nenviada = true" as
NotificacaoEnviada
state "NotificacaoFalhou\nerro = falha no envio" as
NotificacaoFalhou
state "AvaliacaoPendente\nstatus = aguardando" as
AvaliacaoPendente
state "AvaliacaoConcluida\navaliacaoRegistrada = true" as
AvaliacaoConcluida
state "CupomAtivo\nstatus = ativo" as CupomAtivo
```

```
state "CupomExpirado\nstatus = expirado" as CupomExpirado
state "CupomUtilizado\nstatus = utilizado" as CupomUtilizado
```

```
ClienteNaoCadastrado --> ClienteCadastrado : cadastrar(dto)
/ salvarCliente()
ClienteCadastrado --> ClienteCadastrado : desativarConta() /
reativarConta()
ClienteCadastrado --> ClienteAutenticado : login(email,
senha) / gerarTokenJWT()
ClienteAutenticado --> ClienteCadastrado : logout() /
invalidarToken()
```

```
ClienteAutenticado --> AgendamentoPendente :
realizarAgendamento() / bloquearHorario()
AgendamentoPendente --> AgendamentoConfirmado : confirmar()
[horarioDisponivel == true] / enviarNotificacao()
AgendamentoConfirmado --> AgendamentoCancelado : cancelar()
[antecedencia >= 2h] / liberarHorario()
AgendamentoPendente --> AgendamentoCancelado : cancelar()
[antecedencia >= 2h]
AgendamentoConfirmado --> AgendamentoConfirmado :
reagendar(novaDataHora) / liberarHorarioAntigo()
AgendamentoConfirmado --> AgendamentoConcluido :
registrarAtendimento()
```

```
AgendamentoConcluido --> AvaliacaoPendente :
notificarCliente()
AvaliacaoPendente --> AvaliacaoConcluida : avaliar(nota,
comentario) [statusAtendimento == concluido]
```

```
AgendamentoConfirmado --> CupomAtivo : aplicarCupom()
CupomAtivo --> CupomExpirado : verificar() [dataValidade <
hoje]
CupomAtivo --> CupomUtilizado : aplicar() [cupomValido ==
true] / marcarComoUtilizado()
```

```
AgendamentoConfirmado --> NotificacaoPendente :
dispararNotificacao()
NotificacaoPendente --> NotificacaoFalhou : enviar()
[smtpDisponivel == false]
NotificacaoFalhou --> NotificacaoPendente : retry()
[tentativas < 3] / tentativas++
```

```
NotificacaoPendente --> NotificacaoEnviada : enviar()
[smtpDisponivel == true]
NotificacaoFalhou --> [*] : descartar() [tentativas >= 3]
NotificacaoEnviada --> [*]
AvaliacaoConcluida --> [*]
CupomUtilizado --> [*]
CupomExpirado --> [*]
AgendamentoCancelado --> [*]
@enduml
```

8. MODELO ENTIDADE-RELACIONAMENTO

```
@startuml
title Modelo Entidade-Relacionamento - BeautyBook

entity "USUARIO" as USUARIO {
    * id_usuario : BIGINT <<PK>>
    --
    nome : VARCHAR(100)
    email : VARCHAR(100)
    senha_hash : VARCHAR(255)
    tipo : VARCHAR(20)
    data_cadastro : DATE
    ativo : BOOLEAN
}

entity "CLIENTE" as CLIENTE {
    * id_cliente : BIGINT <<PK, FK>>
    --
    telefone : VARCHAR(20)
    cpf : VARCHAR(14)
    endereco : VARCHAR(200)
}

entity "FUNCIONARIO" as FUNCIONARIO {
    * id_funcionario : BIGINT <<PK, FK>>
    --
    especialidade : VARCHAR(100)
    telefone : VARCHAR(20)
```

```

    data_admissao : DATE
}

entity "ADMINISTRADOR" as ADMINISTRADOR {
    * id_administrador : BIGINT <<PK, FK>>
    --
    cargo : VARCHAR(50)
}

entity "SERVICO" as SERVICO {
    * id_servico : BIGINT <<PK>>
    --
    nome : VARCHAR(100)
    descricao : TEXT
    duracao_minutos : INTEGER
    preco : DECIMAL(10,2)
    ativo : BOOLEAN
}

entity "CUPOM" as CUPOM {
    * id_cupom : BIGINT <<PK>>
    --
    codigo : VARCHAR(20)
    desconto_percentual : DECIMAL(5,2)
    data_validade : DATE
    uso_maximo : INTEGER
    uso_atual : INTEGER
    ativo : BOOLEAN
}

entity "AGENDAMENTO" as AGENDAMENTO {
    * id_agendamento : BIGINT <<PK>>
    --
    data_hora : TIMESTAMP
    status : VARCHAR(20)
    valor_total : DECIMAL(10,2)
    valor_desconto : DECIMAL(10,2)
    observacoes : TEXT
    data_criacao : TIMESTAMP
    id_cliente : BIGINT <<FK>>
    id_funcionario : BIGINT <<FK>>
    id_servico : BIGINT <<FK>>
}

```

```
    id_cupom : BIGINT <<FK>>
}
```

```
entity "NOTIFICACAO" as NOTIFICACAO {
    * id_notificacao : BIGINT <<PK>>
    --
    mensagem : TEXT
    tipo : VARCHAR(10)
    data_envio : TIMESTAMP
    enviada : BOOLEAN
    tentativas : INTEGER
    id_agendamento : BIGINT <<FK>>
}
```

```
USUARIO ||--o| CLIENTE : "é um"
USUARIO ||--o| FUNCIONARIO : "é um"
USUARIO ||--o| ADMINISTRADOR : "é um"
```

```
CLIENTE ||--o{ AGENDAMENTO : "realiza"
FUNCIONARIO ||--o{ AGENDAMENTO : "executa"
SERVICO ||--o{ AGENDAMENTO : "inclui"
CUPOM ||--o{ AGENDAMENTO : "aplica"
AGENDAMENTO ||--o{ NOTIFICACAO : "dispara"
@enduml
```